Graph-Based Credit Card Fraud Detection: Comparing Graph Neural Networks and XGBoost Under Random and Temporal Evaluation

By: Tarunvir Singh

10/31/2026

1. Abstract

Most fraud detection systems score one transaction at a time, using things like the amount, the time and the merchant category. That misses the relational side of fraud, like a card that suddenly shows up at a merchant its owner has never used. I compared XGBoost, a standard tabular model, against a graph neural network (GNN) built on a customer–merchant transaction graph. I tested both on a simulated Kaggle dataset of 555,719 transactions (0.39% fraud), once with a random split and once with a temporal split that trains on earlier transactions and tests on later ones. I tuned XGBoost and gave both models comparable features. Tuned XGBoost won on both splits. On the random split its PR-AUC was 0.870, while the GNN averaged 0.663 over five seeds (0.496 to 0.708). On the temporal split XGBoost scored 0.700, while the GNN scored 0.420 in the main run and between 0.353 and 0.487 across seeds. I also tested two assumptions about the graph. Letting messages flow in both directions did not help, and letting the temporal graph include test-period connections made no clear difference. The GNN did catch some fraud that XGBoost missed, but it also raised far more false alarms at the same threshold, so I can't say the two models pick up different signals. Because the data is simulated, these results compare the two setups on this dataset. They do not show how either would perform at a real bank.

2. Introduction

Credit card fraud costs a lot of money every year, and it is hard to detect because fraud is rare (well under 1% of transactions) but expensive to miss, and labeled fraud examples are scarce, a problem Jing et al. take on directly. Most real systems use gradient-boosted trees like XGBoost (Chen and Guestrin). These models look at a transaction's own features, such as the amount, time and category, and sometimes some history for that card. Each transaction gets scored on its own.

A lot of fraud has a relational shape, though. A stolen card might show up at a merchant its owner has never used, or several unrelated accounts might all start buying from the same merchant in a short window. You can't see that by looking at one row at a time. You can see it if you build a graph, with customers and merchants as nodes and transactions as the edges between them. Graph neural networks are designed to learn from that kind of structure, because each node's representation is built from the nodes it connects to (Hamilton et al.). Researchers have started applying them to card fraud (Liu et al.; Sultana et al.; Cherif et al.).

This is a supervised binary classification problem: predict is_fraud (0 or 1) for each transaction. Both models output a fraud probability, which I turn into a yes/no call with a 0.5 threshold. My question was: does a GNN on a transaction graph beat XGBoost on tabular features, does the answer hold up under both a random split and a temporal split, and does the way the graph is wired matter? I tested two wiring choices: whether messages should flow in both directions (Section 5.2), and whether letting the temporal graph include test-period connections inflates the results (Section 5.3).

I gave both models comparable information, so the comparison is as fair as I could make it. They still differ in architecture and in how much tuning each received, so the results compare these two setups rather than isolating the effect of graph structure by itself. The temporal split matters because a real system has to score tomorrow's transactions, not a random sample of yesterday's. Wang et al. also treat time as central, building it into the graph itself.

3. Background

Gradient-boosted trees. Chen and Guestrin introduced XGBoost, a fast and scalable tree-boosting method. It is widely used on tabular data like this. Its limit is that each transaction is scored on its own, with no built-in way to use how it relates to anything else.

GraphSAGE. Hamilton et al. introduced GraphSAGE, which builds a node's embedding by sampling and combining information from its neighbors. This lets it handle nodes it never saw during training, which suits a setting where new customers and merchants keep appearing. I used PyTorch Geometric's SAGEConv layer, which implements this idea.

GNNs for fraud. Many papers apply graph models to credit card fraud. Liu et al. build a weighted multi-graph over transactions and apply GraphSAGE with attention. Alarfaj and Shahzadi combine GNNs with autoencoders for real-time bank fraud detection, and Jing et al. use a few-shot GNN for the problem of scarce fraud labels. Sultana et al. evaluate a GNN-based pipeline called detectGNN, and Cherif et al. use an encoder–decoder GNN. Wang et al. go further and build time into the graph itself, using a temporal heterogeneous graph with contrastive learning. Earlier neural approaches also targeted speed, such as the parallel granular networks of Syeda et al. Together these papers suggest graph structure can help, though the results depend on the dataset and on how the graph is built. My project is simpler than most of these. I follow the general recipe of Liu et al. and Jing et al., which is to build a transaction graph, run an inductive GNN, and compare against a tabular baseline. The addition is that I test on a later-in-time split as well as a random one.

4. Dataset

I used the Credit Card Transactions Fraud Detection Dataset from Kaggle (Shenoy), which was generated with the Sparkov simulator. I worked with the fraudTest.csv file: 555,719 transactions from June 21 to December 31, 2020, covering 924 customers and 693 merchants. The fraud rate is 0.386% (2,145 fraudulent transactions). Because the data is simulated, the fraud patterns come from the generator's rules rather than from real criminals.

Cleaning. I dropped identifying columns that aren't useful as features (name, street address, transaction ID). I split the timestamp into hour, day of week and month, computed age from date of birth, and computed the customer–merchant distance from their coordinates with the haversine formula. Category was one-hot encoded into 14 columns. There were no missing values or duplicates.

Feature engineering. For fairness, I gave XGBoost aggregate features per card (transaction count, average amount, number of distinct merchants) and per merchant (transaction count, historical fraud rate). The GNN gets the same statistics as node features. All of them were computed from the training portion of whichever split was in use.

Splits. Two of them, matching the research question:

- Random split: 80/20, stratified by fraud label — 444,575 training transactions, 111,144 test.
- Temporal: everything before December 7, 2020 for training (444,575 transactions) and everything after for testing (111,144). The fraud rate drops from 0.44% in training to 0.17% in testing, so this split has a real distribution shift in it.

Figure 1 shows the class imbalance and the monthly fraud rate. The drop in the later months is what motivated the temporal split. Figure 2 shows that fraud amounts cluster around $250 to $350, with a smaller cluster of very cheap transactions.

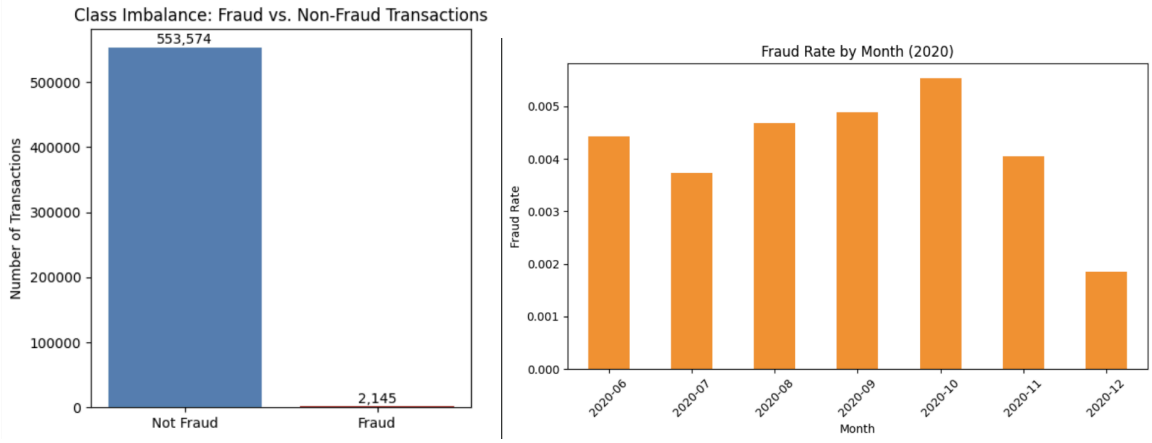


Figure 1. Left: class imbalance, with 553,574 non-fraud transactions against 2,145 fraud. Right: fraud rate by month, which falls in the last months of 2020.

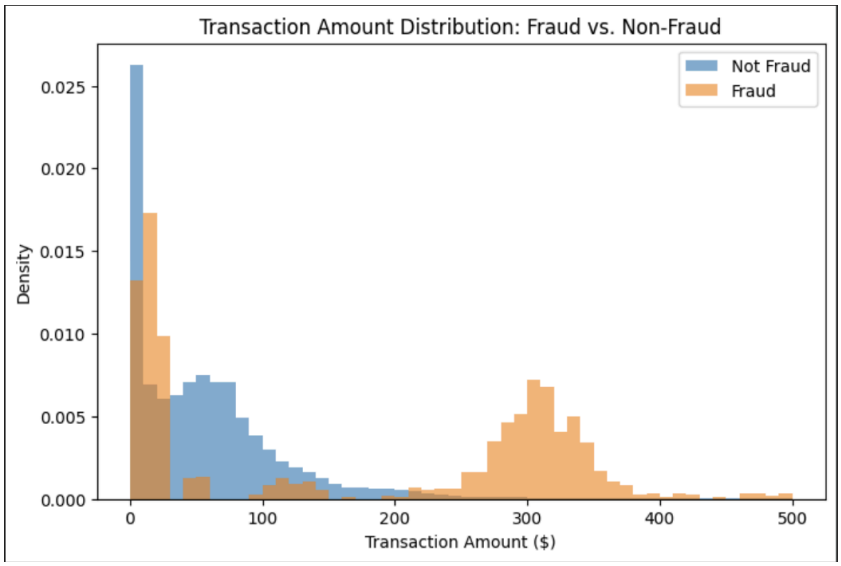


Figure 2. Transaction amounts for fraud and non-fraud. Fraud clusters around $250 to $350 and also overlaps with legitimate transactions at lower amounts.

Data Set Link - https://www.kaggle.com/datasets/kartik2112/fraud-detection

5. Methodology / Models

XGBoost baseline. I trained an XGBClassifier, the implementation of Chen and Guestrin's method, on a flat table of amount, time features, age, customer–merchant distance, city population, gender, one-hot category, and the engineered card and merchant statistics. Fraud is only 0.39% of the data, so I set scale_pos_weight to the training set's negative-to-positive ratio (about 258 on the random split and about 226 on the temporal split). Without that the model would mostly predict "not fraud."

5.1 Hyperparameter Tuning

I tuned XGBoost on a validation set taken from the training data (20%, stratified), not on the test set, so the reported numbers aren't inflated by tuning on the data I evaluate on.

I started with a random search of three combinations of max depth, number of trees and learning rate:

Trial	**max_depth**	**n_estimators**	**learning_rate**	**Validation PR-AUC**
1	4	100	0.2	0.8552
2	8	100	0.05	0.8447
3	6	300	0.05	0.8895

Then I ran a grid search over max depth from 2 to 10, holding 200 trees and a learning rate of 0.1 fixed (Figure 3):

max_depth	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Validation PR-AUC	0.666	0.757	0.859	0.885	0.898	0.907	0.913	0.917	0.909

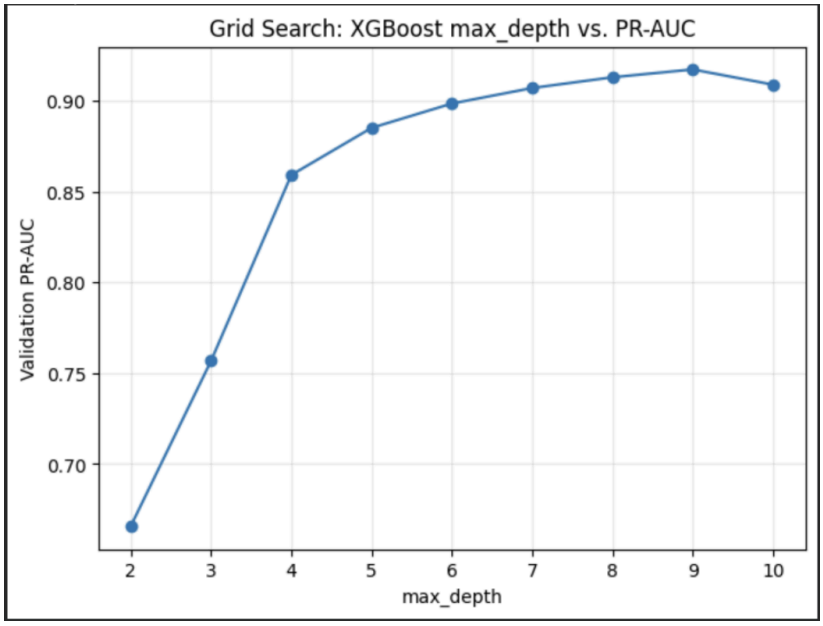


Figure 3. Validation PR-AUC as max_depth goes from 2 to 10, with 200 trees and learning rate 0.1. It peaks at depth 9 and dips at 10.

Depth 9 scored 0.917 on validation, higher than any random-search trial, so I used it. The final model uses depth 9, 200 trees and a learning rate of 0.1, retrained on the full training set. Only max depth was tuned this way. The tree count and learning rate came from my initial setup, and the random search was small. The validation split was random rather than time-ordered, and I reused the same depth for the temporal experiment.

5.2 The GNN

The graph has 1,617 nodes (924 customers and 693 merchants) and one edge per transaction from a customer to a merchant. The edge features are nearly the same per-transaction features XGBoost gets (amount, hour, day, month, age, distance and category), although the GNN does not receive gender or city population. Customer nodes carry average amount, transaction count and distinct-merchant count. Merchant nodes carry transaction count, historical fraud rate and average amount.

The model has two SAGEConv layers (the GraphSAGE layer of Hamilton et al.) that produce node embeddings, with 64 hidden units and dropout of 0.3, followed by a small feedforward classifier. For each transaction it takes the customer embedding, the merchant embedding and the edge features, and predicts fraud. I trained for 200 epochs with Adam (learning rate 0.005, weight decay 5e-4) and a class-weighted cross-entropy loss that matches the imbalance correction used for XGBoost. I trained a fresh model for each split and rebuilt the features, scalers and masks for each one from training data only. Without standardizing the

features, training was unstable. I set a random seed before each run, but GPU results still vary slightly from run to run, so I report several seeds where I can.

Directed vs. undirected message passing. The transaction edges point one way, customer → merchant, and SAGEConv only aggregates along incoming edges. With directed edges only, merchants hear from their customers but customers hear from no one. I assumed that would hurt, so my main model passes messages over an undirected copy of the graph. A mentor comment on an earlier version asked whether the topology really contributes, so I tested it directly. I trained the same GNN for 200 epochs on the random split with each setup, using five seeds each. Directed-only reached a mean PR-AUC of 0.652 (standard deviation 0.119) and undirected reached 0.663 (standard deviation 0.083). The 0.011 gap is tiny compared with the seed-to-seed spread, and in each setup one seed did much worse than the rest (0.416 directed, 0.496 undirected). So I found no detectable benefit from undirected message passing, and my original reasoning did not hold up. A likely reason is that the edge features already carry most of the useful signal.

A correction. An earlier version of this evaluation had a bug. The train and test masks for the random-split GNN were built from row labels that were reset when I merged the engineered features, so the test set was a subset of the training set. I caught this by checking that the two masks did not overlap, fixed it, and reran every random-split GNN result. All GNN random-split numbers in this paper come from the corrected split. The XGBoost results and the temporal results were not affected.

5.3 Leakage Check for the Temporal Graph

In the temporal experiment, the node features, scalers, labels and masks all come from the training period only. But the graph that SAGEConv passes messages over originally included every transaction, including those after the December 7 cutoff, so it carried some information about who transacts with whom in the test period. To see whether that mattered, I retrained the temporal GNN under two conditions with three seeds each (0, 1, 2): message passing over the full graph, and message passing over only the training-period connections.

Mean PR-AUC was 0.440 with the full graph (runs of 0.353, 0.480 and 0.487) and 0.431 with the training-period graph only (runs of 0.379, 0.441 and 0.472). The 0.009 gap is much smaller than the spread between seeds (standard deviations of 0.061 and 0.039), so I found no detectable effect. With three seeds this rules out a large effect but not a small one. The main temporal results use the full graph.

6. Results and Discussion

6.1 Quantitative results

PR-AUC is my main metric. With 0.39% fraud, a model that always says "not fraud" would be 99.6% accurate, so accuracy tells you nothing. A random ranking would score a PR-AUC equal to the fraud rate, about 0.39% on the random test set and about 0.17% on the temporal one. Precision and recall use a 0.5 threshold for both models.

Model	Split	PR-AUC	ROC-AUC	Recall (fraud)	Precision(fraud)
XGBoost (tuned)	Random	0.870	0.991	0.825	0.839
XGBoost (tuned)	Temporal	0.700	0.972	0.668	0.724
GNN(5 - seed mean)	Random	0.663	0.988	0.941	0.076
GNN(main run)	Temporal	0.420	0.982	0.815	0.070

The XGBoost rows are one deterministic run each. The random-split GNN row is the mean of five seeds (PR-AUC standard deviation 0.083, range 0.496 to 0.708). The temporal GNN row is a single seeded run, and across three seeds its PR-AUC ranged from 0.353 to 0.487 (Section 5.3). Figures 4 to 6 use the first seed on the random split.

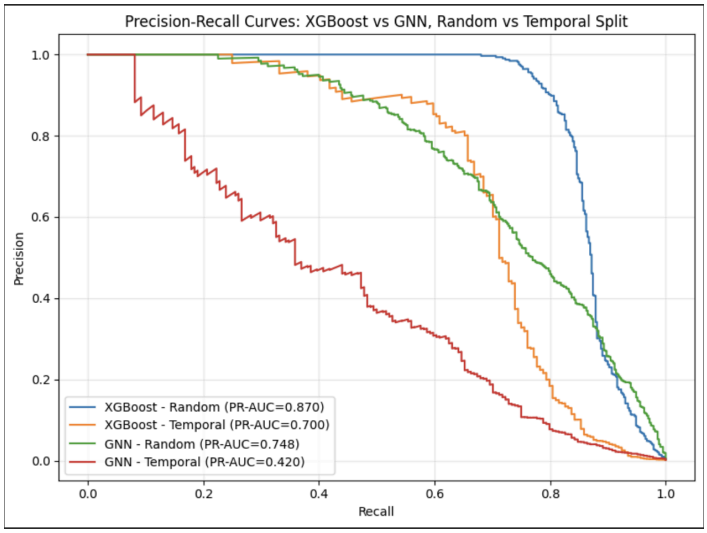


Figure 4. Precision-recall curves for all four model and split combinations. Both models' temporal curves sit well below their random-split curves.

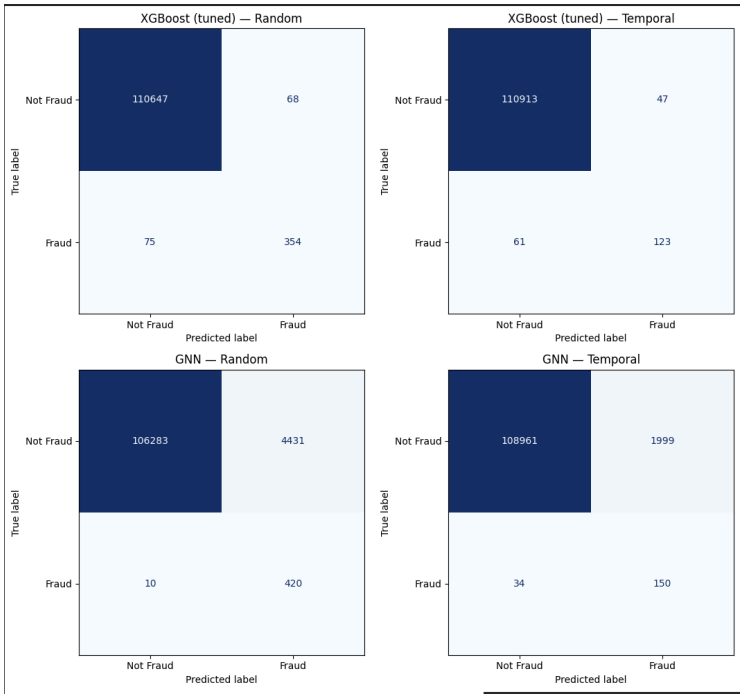


Figure 5. Confusion matrices at a 0.5 threshold. The GNN flags far more legitimate transactions than XGBoost, which is why its precision is low.

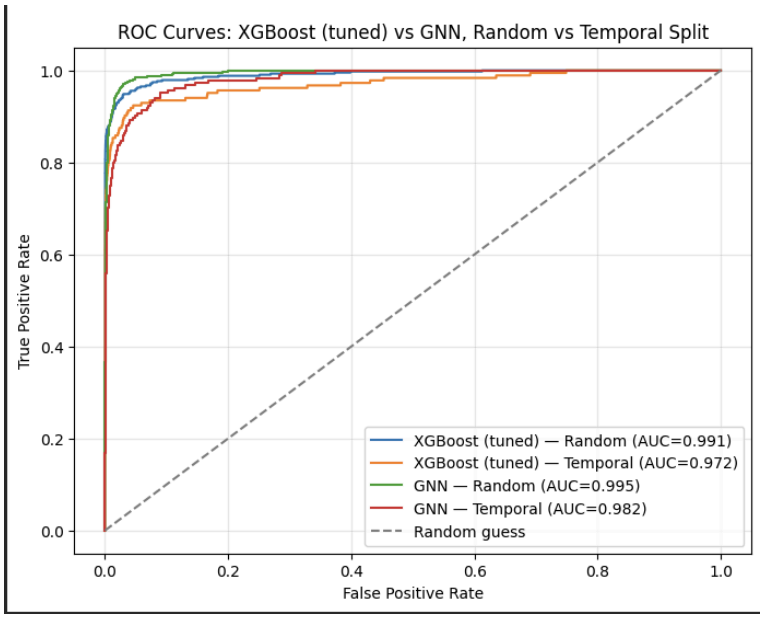


Figure 6. ROC curves for all four combinations. ROC-AUC stays high (0.972 to 0.991) because it is not sensitive to class imbalance the way PR-AUC is.

On the random split, tuned XGBoost beats the GNN, 0.870 to 0.663 on average, with a much better balance of precision and recall. XGBoost's 0.870 is above every one of the five GNN runs, whose best was 0.708. The GNN catches nearly everything (0.941 recall) but flags a lot of legitimate transactions to do it. Its precision is 0.076, and in the first-seed run it produced 5,517 false positives against 68 for XGBoost (Figure 5).

On the temporal split both models get worse, which makes sense because some of what they learned on the random split probably doesn't hold over time. XGBoost's PR-AUC drops 19.5% (0.870 to 0.700). The GNN's mean drops about 34%, from 0.663 on the random split (five seeds) to 0.440 on the temporal split (three seeds). Its temporal PR-AUC ranged from 0.353 to 0.487 across those seeds, so the exact drop is approximate. Still, XGBoost's 0.700 is above every one of those GNN runs, so its lead on the temporal split doesn't depend on the seed.

This was not my original result. Before I tuned XGBoost, the GNN's relative drop looked smaller than XGBoost's, so it looked more robust to temporal shift. Once XGBoost was tuned, that advantage went away. A new model can't be said to beat a baseline on some property until the baseline has been tuned properly.

6.2 Qualitative analysis

I compared the two models on individual random-split test transactions, using tuned XGBoost and the first-seed GNN, with a 0.5 threshold for both. Of the 429 fraud cases, both models caught 345, the GNN alone caught 56, XGBoost alone caught 9, and both missed 19. So the GNN caught 56 frauds that XGBoost missed, while XGBoost caught 9 that the GNN missed.

This comparison needs a caution. At a 0.5 threshold the GNN flags far more transactions than XGBoost, with 5,517 false positives against 68. Part of why it catches more fraud is simply that it raises more alarms, so these counts don't show that it sees different signal.

I tested two ideas for what the GNN-only catches have in common. The first was that the GNN catches smaller transactions. The median amount of the GNN-only catches was $ 49.38, far below the $ 498.78 median for all fraud. But XGBoost's nine unique catches had an even lower median ($12.54), so small amounts don't separate the models. The second idea was that the GNN catches fraud at merchants with no fraud history. Of the 56 GNN-only catches, 29 were at merchants with exactly 0% training fraud rate. These cases also involved somewhat busier cards: a median of 597.5 transactions and 388 distinct merchants, against 499.5 and 338 for the typical customer. I didn't run a test that isolates either factor, so this describes the cases and doesn't explain them.

6.3 Limitations

The data is simulated, so everything here is a comparison within that environment and not a claim about real banking data. It is also one dataset covering six months, and the same customers and merchants appear in both the training and test periods. The graph is simple: one bipartite structure with three hand-built features per node type. A richer design, such as the weighted multi-graph of Liu et al. or the temporal heterogeneous graph of Wang et al., could change the comparison. I tried only one GNN architecture, and I set its hyperparameters during development without a formal search, unlike XGBoost. GNN scores vary a lot by seed (0.496 to 0.708 on the random split and 0.353 to 0.487 on the temporal split). My directed-vs-undirected experiment used five seeds and my leakage experiment only three, so they can rule out large differences but not small ones. The merchant fraud rate feature is computed on training rows including each row's own label, which can make it look more useful in training than it is at test time. I used a random validation split to tune XGBoost, even for the temporal experiment. Finally, I judged both models at a single 0.5 threshold and did not compare them at a matched number of false alarms, so I can't say whether the GNN's high recall would be useful in practice.

7. Conclusion

I asked whether a GNN on a transaction graph beats XGBoost for fraud detection, under both a random and a temporal split. Once XGBoost was tuned, the tuned XGBoost configuration outperformed the tested GNN configuration on both splits, and it dropped less under temporal shift. That is a less exciting answer than "the graph model wins," but it is the one the evidence supports. The main lesson is that an untuned baseline can make a fancier model look better than it is, which is what happened here before the tuning pass.

The GNN still caught some fraud that XGBoost missed (56 cases, against 9 going the other way), and I couldn't trace that to any single feature. But at a 0.5 threshold it also raised about 80 times as many false positives, so this is not clear evidence that it sees something XGBoost can't. The first thing to test is both models at a matched number of false alarms. If the GNN's extra catches hold up, that would be a reason to combine the two, as Alarfaj and Shahzadi do by pairing a GNN with autoencoders. For example, I could average their scores and check the blend on held-out data, which I did not test. Other next steps are a richer graph (transaction-to-transaction links or typed nodes, like Wang et al.), an attention-based GNN as in Liu et al., a real hyperparameter search for the GNN, extra fields a real bank would have such as device or card-BIN information, and running the comparison on the full multi-year dataset.

8. Acknowledgements

Thanks to my Inspirit AI mentor, Mikhail, for his guidance throughout this project, and to the Inspirit AI program for the mentorship opportunity that made this possible.

9. Works Cited

G. Liu, J. Tang, Y. Tian and J. Wang, "Graph Neural Network for Credit Card Fraud Detection," *2021 International Conference on Cyber-Physical Social Intelligence (ICCSI)*, Beijing, China, 2021, pp. 1-6, doi: 10.1109/ICCSI53130.2021.9736204.

F. Khaled Alarfaj and S. Shahzadi, "Enhancing Fraud Detection in Banking With Deep Learning: Graph Neural Networks and Autoencoders for Real-Time Credit Card Fraud Prevention," *IEEE Access*, vol. 13, pp. 20633-20646, 2025, doi: 10.1109/ACCESS.2024.3466288.

R. Jing, H. Tian, G. Zhou, X. Zhang, X. Zheng and D. D. Zeng, "A GNN-based Few-shot learning model on the Credit Card Fraud detection," *2021 IEEE 1st International Conference on Digital Twins and Parallel Intelligence (DTPI)*, Beijing, China, 2021, pp. 320-323, doi: 10.1109/DTPI52967.2021.9540093.

I. Sultana, S. M. Maheen, N. Kshetri and M. N. Fardous Zim, "detectGNN: Harnessing Graph Neural Networks for Enhanced Fraud Detection in Credit Card Transactions," *2025 13th International Symposium on Digital Forensics and Security (ISDFS)*, Boston, MA, USA, 2025, pp. 1-6, doi: 10.1109/ISDFS65363.2025.11011957.

Kaggle. "Credit Card Transactions Fraud Detection Dataset" (kartik2112), generated via the Sparkov Data Generation tool.

W. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.

M. Syeda, Yan-Qing Zhang and Yi Pan, "Parallel granular neural networks for fast credit card fraud detection," 2002 IEEE World Congress on Computational Intelligence. 2002 IEEE International Conference on Fuzzy Systems. FUZZ-IEEE'02. Proceedings (Cat. No.02CH37291), Honolulu, HI, USA, 2002, pp. 572-577 vol.1, doi: 10.1109/FUZZ.2002.1005055. keywords:

Cherif, A., Ammar, H., Kalkatawi, M. *et al.* Encoder–decoder graph neural network for credit card fraud detection. *J. King Saud Univ. Comput. Inf. Sci.* 36, 102003 (2024). https://doi.org/10.1016/j.jksuci.2024.102003

J. Wang, J. Liu, W. Zheng and Y. Ge, "Temporal Heterogeneous Graph Contrastive Learning for Fraud Detection in Credit Card Transactions," in *IEEE Access*, vol. 13, pp. 145754-145771, 2025, doi: 10.1109/ACCESS.2025.3599787.

The full implementation (preprocessing, feature engineering, model training and evaluation) is in this Colab notebook:

Code Availability - Graph Neural Networks and XGBoost Code